

AMOS: A Memory Operating Layer for Autonomous Data Analysis

Abstract

Companies do not need another stateless analytics chatbot. They need a persistent analytical operating system that sits over the company’s evolving memory, learns how the business defines and analyzes itself, and automates the complete data-analysis loop from request or signal to verified deliverable. This company memory spans data and schemas, metrics and business ontology, policies and authority, prior incidents and decisions, reviewer corrections, execution history, and output conventions. It is durable and governed outside any one model; each task receives only a bounded, permission-safe active context.

This proposal introduces AMOS, a model-neutral analytical operating layer between company memory, analytical applications, and the systems they use. AMOS externalizes analytical state as typed, versioned memory; constructs task-specific context; plans and executes SQL, Python, statistical, visualization, and reporting work; verifies results; preserves human review; and revisits prior conclusions when governing evidence changes. A model such as Google Gemma 4 may serve as one replaceable execution engine. The model is not the operating system, and conversational data querying—including a Databricks Genie-style experience—is one application on top of AMOS rather than the product boundary.

The full product is a platform on which internal teams and external developers build business-review applications, anomaly-response agents, forecasting workflows, planning tools, finance-reporting agents, operational monitors, and vertical analysis systems. The paper-sized research contribution validates its most important substrate: a governed memory layer and analytical state transaction (A-TXN) that connect persistent state, bounded context, execution, publication, claim dependencies, replay, and invalidation. We will compare that substrate with matched agents and a production-faithful composed governance stack under controlled schema, metric, data, document, and policy churn, with explicit acceptance and rejection criteria.

1 Motivation and Problem

An analyst’s job is not a single prompt-to-SQL call. It includes understanding a business objective, discovering and assessing data, selecting methods, executing tools, checking uncertainty and quality, producing artifacts, incorporating review, and monitoring for change. AMOS is designed to automate this end-to-end loop continuously while retaining the company’s accumulated context between tasks.

Correctness depends on more than model reasoning. At time t , a task observes independently changing state:

$$\mathcal{T}_t = (q_t, D_t, C_t, M_t, X_t, H_{<t}, F_{<t}, P_t), \quad (1)$$

where q_t is the request, D_t the data snapshot or stream state, C_t schema and catalog state, M_t metric and semantic state, X_t governed documents, $H_{<t}$ prior work, $F_{<t}$ reviewed feedback, and P_t identity and policy.

These systems rarely share a transaction manager. Between retrieval and publication, a metric may change, permission may be revoked, a table may migrate, or a watermark may advance. A fluent answer can therefore be state-invalid even when its SQL ran successfully. Prompt expansion and ordinary retrieval do not solve the time-of-check/time-of-use problem; job-level lineage records execution but not necessarily which claim depended on which governing state.

The research problem is:

Can a governed memory operating layer over heterogeneous analytical state materially reduce state-related failures and audit cost for continuous data analytics agents, compared with conventional agent orchestration and a composed governance stack, without unacceptable latency, false rejection, or operational complexity?

The intended output is not only an artifact Y_t , but a validated state observation V_t , computation E_t , claim dependency graph $\mathcal{G}_t$, and replay descriptor R_t :

$$\mathcal{T}_t \rightarrow (Y_t, V_t, E_t, \mathcal{G}_t, R_t). \quad (2)$$

2 Product North Star and Platform Boundary

2.1 Three Product Identities

AMOS has three identities that share one operating substrate:

1. **Company analytical memory.** A versioned, governed, executable representation of how the company understands its data—not merely a vector database of documents.
2. **Data-analysis agent runtime.** The environment for context construction, planning, model invocation, SQL and Python execution, verification, repair, artifact generation, review escalation, memory updates, scheduling, and recovery.
3. **Autonomous analytics workforce.** Persistent applications that answer questions, explore data, investigate anomalies, monitor metrics and quality, forecast, evaluate experiments, build reports and dashboards, and update conclusions as evidence changes.

This makes the product category explicit: AMOS is the operating layer that turns governed company state into continuously executed, verified, replayable analysis. Models are processors used by the layer; warehouses and lakehouses remain systems of record; user experiences and domain agents are applications above it.

2.2 The Company’s Analytical Constitution

As AMOS observes work and receives governed corrections, it builds a structured analytical constitution. Every learned item carries scope, authority, effective time, permissions, provenance, confidence, and supersession state.

Constitution layer	Company-specific memory learned by AMOS
Business ontology	Customer, account, active user, order, subscription, region, product, campaign, incident, aliases, and relationships.
Metrics	Names, formulas, filters, owners, effective dates, variants, allowed dimensions, and known limitations.
Analytical conventions	Excluded populations, confidence levels, reporting calendars, fiscal periods, attribution rules, minimum samples, and accepted comparison methods.
Organizational authority	Who defines each metric, approves reports, owns each dataset, and whose correction overrides a prior assumption.
Historical knowledge	Prior incidents, recurring seasonal effects, known defects, accepted explanations, rejected hypotheses, and executive decisions.
Output preferences	Report formats, chart styles, detail level, required reviewers, escalation thresholds, and audience-specific language.

Table 1: The analytical constitution turns generic model capability into durable, company-specific analytical behavior.

Human feedback is therefore a governed state transition, not a note appended to a prompt. A correction such as “exclude internal accounts” is stored with its owner, scope, effective interval, supporting decision, and downstream dependencies so it can affect the next report without silently rewriting unrelated work.

2.3 Applications Built on AMOS

Internal teams and external developers build analytical applications against stable memory, execution, verification, and artifact interfaces. Representative applications include business-review apps, anomaly-response agents, forecasting workflows, planning tools, finance-reporting agents, operational monitoring, experiment-analysis systems, and vertical-specific analytical products. These applications share the same company constitution and execution history instead of rebuilding context and governance independently.

A natural-language data assistant can ask questions, generate queries, and explain results. That capability is useful but deliberately subordinate: a Databricks Genie-style interface is one side quest among many. The defining product behavior is that AMOS preserves and governs what the company knows, runs analytical work repeatedly, learns from review, and re-evaluates outputs when their dependencies change.

2.4 Model-Neutral Execution

AMOS may route tasks to different model classes: a small local model for memory classification, an embedding model for entity matching, a coding-capable model for SQL or Python, a reasoning model for investigation planning, a multimodal model for chart inspection, and deterministic code for verification. A customer-hosted Google Gemma 4 model is one possible

engine, particularly for controlled or on-premise deployments. Models can be upgraded or replaced without losing metric knowledge, schema mappings, reviewer corrections, execution history, provenance, scheduled work, or policies.

2.5 Compounding Product Moat

The durable advantage is not access to a particular model. It compounds through six assets: (1) company-specific analytical memory accumulated across years of definitions, corrections, and decisions; (2) execution history containing successful plans, failed queries, useful segmentations, and reviewer behavior; (3) governed feedback that survives the task in which it was given; (4) an analytical dependency graph linking claims, metrics, data, documents, code, decisions, and reports; (5) connectors that understand versions, permissions, schema change, and source semantics; and (6) a reliable runtime that repeats, recovers, validates, and preserves work across model upgrades.

2.6 Scope

The product vision and the research contribution are intentionally different in breadth. The product goal is a complete analytical operating system and application platform. The proposed paper-sized contribution validates its substrate: the typed memory model and the A-TXN consistency, context-selection, commit, and invalidation protocol. SQL verification, access control, and provenance storage support that memory contract. The prototype targets read-only analytical SQL, governed documents, charts, and report claims. Arbitrary notebook code, general multi-agent scheduling, autonomous writes to production data, and full LLM trajectory determinism are outside the primary research evaluation, not outside the long-term product.

Raw data remains in its systems of record. AMOS stores versioned metadata, compact summaries, policy-safe references, execution records, and dependency edges. It is not a new warehouse or universal enterprise knowledge store.

3 Startup Build Strategy

The startup should earn the right to build the full platform through a narrow, measurable sequence. The first commercial wedge is recurring operational analysis for data teams that already have a warehouse, defined business metrics, and painful manual review. AMOS begins as a read-only control layer for one high-value workflow, proves that it prevents stale or unsupported analysis, and then expands into a shared runtime for many analytical applications.

3.1 Ideal First Customer and Initial Wedge

The ideal design partner is a 50–500 person software or commerce company with a modern cloud warehouse, a small centralized data team, and at least one recurring metric review or incident workflow. The initial buyer is the head of data or analytics; the daily users are analytics engineers, analysts, and operational leaders. A suitable first workflow runs at least weekly, costs several analyst-hours per cycle, depends on governed metric definitions, and has an identifiable reviewer.

The recommended wedge is a payment-health or subscription-health review. It is frequent, economically meaningful, bounded enough to verify, and already represented by the prototype scenarios. The startup should not begin with unrestricted natural-language access to every company table. It should begin with one approved metric family, one source system, read-only SQL, a small set of report templates, and a mandatory human publication gate.

3.2 Build Principles

1. **One vertical slice before platform breadth.** Complete request-to-report execution for one workflow before adding many connectors or agent types.
2. **Governance in the execution path.** Identity, permissions, versions, verification, and provenance are runtime controls, not optional metadata added after generation.
3. **Human approval before autonomy.** Early releases recommend, explain, and package evidence. Autonomous publication is enabled only for task classes with measured reliability and an explicit rollback path.
4. **Deterministic core, replaceable models.** Policy checks, version validation, storage, and evidence capture stay outside model prompts. Model providers can change without erasing company memory.

5. **Every phase has an exit gate.** A phase advances only when customer value, correctness, security, and operability meet predefined thresholds.

3.3 Step-by-Step Product Roadmap

Step	Product	Build scope	Exit gate
0: Foundation	Reproducible local core	Typed memory objects, migrations, tenant/identity model, read-only tool boundary, event log, CI, observability, and fixture-backed demo.	One-command setup; tests pass; no secret or cross-tenant data in traces; every demo claim resolves to evidence.
1: Governed answer	Design-partner alpha	One warehouse connector, one metric family, permission-first retrieval, SQL planning, schema/metric verifier, cited answer, and reviewer approval.	At least 90% safe-and-correct completion on the frozen customer task set; zero confirmed permission leaks; median reviewer time falls by 30%.
2: Persistent analyst	Private beta	Feedback memory, report and chart artifacts, replay packages, scheduling, freshness checks, and change notifications for the first workflow.	Four consecutive weekly cycles complete without silent state-invalid publication; reviewer corrections affect the next applicable run.
3: Team operating layer	Paid pilot	SSO/RBAC, two additional connectors, audit console, version rollback, incident recovery, usage metering, and service-level objectives.	Three paying design partners; at least two weekly active users per account; 99% successful scheduled runs excluding source outages.
4: Application platform	General availability	Stable APIs/SDK, application templates, model routing, connector kit, tenant administration, retention controls, and deployment automation.	Five production customers; repeatable onboarding under ten business days; gross margin and support load fit the operating plan.
5: Controlled autonomy	Expansion	Policy-scoped auto-publication, dependency-driven re-analysis, portfolio scheduling, advanced statistical tools, and partner-built applications.	Each autonomous task class passes a separate safety case, rollback drill, and customer approval; expansion revenue exceeds new-logo revenue.

Table 2: A startup sequence that turns the AMOS thesis into progressively broader products. Time is secondary to passing each exit gate.

3.4 Minimum Product Architecture

The first deployable system has seven services or modules, even if a small team initially packages them as one process: (1) connector adapters that expose source versions and enforce source credentials; (2) a typed memory store; (3) a policy and identity gate; (4) a context compiler; (5) an agent/tool runner; (6) deterministic verification and publication gates; and (7) an evidence store with replay and audit views. A queue supports scheduled work, while an append-only event log supports recovery and debugging. Raw customer data remains in the warehouse; retained samples require explicit policy.

The first production deployment should favor a modular monolith with PostgreSQL, object storage, a managed queue, and isolated tool workers. This keeps transactions and operations understandable for a small team. Service extraction is justified only by tenant isolation, scaling, or deployment requirements. The public contract is the A-TXN lifecycle, not the internal service topology.

3.5 Team, Metrics, and Operating Cadence

An initial four-to-six person team can cover the wedge: a systems/data engineer for memory and connectors, an agent/runtime engineer, a product engineer for review and audit surfaces, a design-partner lead with analytics depth, and a founder responsible for architecture and customer discovery; security or infrastructure expertise can be fractional until the first paid pilot.

The weekly company scorecard should separate value from safety. Value metrics are analyst-hours saved, time to reviewed answer, weekly completed workflows, reviewer acceptance, and design-partner retention. Safety metrics are state-invalid publication rate, permission incidents, unsupported-claim rate, verifier false rejects, replay success, and mean recovery time. Cost metrics are model cost per accepted artifact, p95 task latency, storage growth, and support hours per account. Growth is not allowed to conceal a regression in permission or publication safety.

Each phase follows the same cadence: discover a customer workflow; freeze its task and policy contract; implement the smallest vertical slice; shadow the existing process; compare results with reviewers; release behind a human gate; instrument failures; and only then generalize reusable primitives. Product requests that do not improve the active phase's exit gate remain out of scope.

3.6 Commercial and Technical Risks

The largest commercial risk is becoming a services-heavy analytics consultancy. The mitigation is a fixed onboarding contract, reusable connector interfaces, and a clear limit on customer-specific logic. The largest product risk is a polished assistant without trusted publication behavior; the mitigation is to sell reviewed workflow completion rather than chat. The largest technical risks are source-version ambiguity, verifier false rejection, permission races, and runaway evidence growth. The roadmap measures each explicitly and narrows supported task classes when a safe contract cannot be enforced.

4 Research Questions and Falsifiable Hypotheses

The proposal separates analytical accuracy, state validity, security, auditability, and cost. It will not use conformance to an AMOS-defined capability checklist as the sole outcome.

	Question	Hypothesis	Disconfirming result
RQ1	Does the AMOS memory layer reduce state-invalid outputs?	H1: governed memory selection plus publish-time validation improves safe-and-correct completion over a matched agent.	The task-resampled 95% interval for the paired improvement includes zero, or benign-task utility falls by more than 5 points.
RQ2	Does a unified memory layer help beyond ordinary composition?	H2: shared typed memory, a version vector, and a commit boundary reduce stale commits and policy races relative to a production-faithful composed stack.	The composed stack is statistically indistinguishable on state-valid completion and race containment at comparable cost.
RQ3	Does claim dependency tracking help reviewers?	H3: claim-level views improve defect detection or reduce audit time without credible degradation on the other outcome.	Neither primary audit outcome improves, or false alarms materially increase.
RQ4	What does the memory layer cost?	H4: control-path overhead stays within 25% of the composed baseline at matched load, with no loss of retrieval recall or throughput collapse as persistent memory grows.	Overhead exceeds the bound without a statistically meaningful correctness or audit benefit.
RQ5	Does enforcement resist realistic attacks?	H5: external capability enforcement yields zero observed permission leaks and lower attack success than prompt-only or post-filter controls.	Any confirmed permission leak occurs, or benign utility is reduced below the preregistered noninferiority margin.

Table 3: Research questions, hypotheses, and explicit rejection conditions.

The memory-layer thesis is supported only if H1 succeeds against both the matched agent and the composed baseline. H3 and H4 support auditability and practicality; they cannot rescue a null result on the central systems thesis.

5 Prior Systems and the Novelty Gap

5.1 Agent Context and Memory

Retrieval-augmented generation supplies non-parametric evidence [2]; long contexts still require selection and can underuse centrally placed information [3]. ReAct and tool-using agents interleave model reasoning with actions [4, 5]. MemGPT explicitly treats prompt context as managed memory [1]; Generative Agents and Reflexion persist observations or verbal feedback [19, 20]. These systems motivate bounded context and durable memory, but do not provide a cross-source analytical version vector or publish-time validation tied to output claims.

5.2 Analytical State and Reproducibility

Stream processors formalize event time, watermarks, state, and replay [9, 10, 11]; lakehouse systems provide transactional table versions [12]. Semantic layers, catalogs, and data-quality systems govern definitions and validate data [15, 16]. Text-to-SQL benchmarks increasingly include enterprise schemas and workflow context [6, 7, 8]. These systems govern their own resources, but an agent task can span several of them without a common observation and publication boundary.

5.3 Provenance, Lineage, and Security

PROV and OpenLineage model entities, activities, datasets, jobs, and runs [13, 14]; noWorkflow and YesWorkflow capture script and workflow provenance [21, 22]. AMOS builds on these representations but proposes a lower-granularity dependency between a published claim and the exact analytical state, computation, and review status that governed it. Indirect prompt injection shows why retrieved content cannot be trusted to enforce its own isolation [17, 18]; AMOS therefore places capability authorization outside the model.

5.4 Why This Is Not Merely Renaming Governance as Memory

No individual AMOS component is claimed as new. The proposed novelty is a unified *memory operating abstraction*: persistent typed analytical memory is projected into bounded active context through a task-scoped transaction that joins (1) the versions and policy under which memory was selected, (2) the capabilities under which tools ran, and (3) the claim dependencies committed at publication. This enables optimistic cross-source validation and claim-level invalidation that ordinary orchestration does not provide automatically.

The decisive comparison is not a weak RAG system. It is a production-faithful stack containing an agent orchestrator, permission-filtered retrieval, semantic definitions, catalog/schema metadata, policy enforcement, OpenLineage-style events, and artifact storage. Both systems receive identical models, tools, sources, budgets, and churn events. The experiment asks whether the explicit A-TXN abstraction adds measurable value beyond careful integration.

System class	Bounded context	Pre-context policy	Cross-source validation	Claim dependencies	Native invalidation unit
Long context / tool agent	Optional	No	No	No	None
Permission-filtered RAG	Yes	Yes	No	No	Retrieved item
Catalog + semantic + lineage stack	Optional	Configurable	Per-system / orchestrated	Usually artifact or job	Dataset, job, artifact
Agent memory system	Yes	Configurable	No	No	Memory entry
AMOS memory layer	Role-bounded	Yes	Optimistic vector validation	Yes	Claim

Table 4: The claimed gap is the common transaction and dependency boundary, not ownership of every component. Entries describe the evaluated configurations, not universal limits on each system class.

6 Technical Core of the AMOS Memory Layer

AMOS separates persistent analytical memory from bounded model-visible context. At time t :

$$\mathcal{M}_t = (M_t^d, M_t^c, M_t^m, M_t^x, M_t^h, M_t^f, M_t^p), \quad (3)$$

where the tiers record data or stream state, schema/catalog state, metric semantics, governed documents, prior analytical history, reviewed feedback, and provenance/policy. Each object carries a stable ID, type, source, authority, effective interval, transaction time, permissions, sensitivity, version, status, supersession edges, and provenance reference. Raw records normally remain in source systems; memory objects hold governed metadata, summaries, and immutable references.

The active context $W_t \subset \mathcal{M}_t$ is a temporary, task-specific projection constrained by identity, time, required roles, and budget. The A-TXN is the privileged protocol by which an agent reads this memory projection, uses scoped tools, validates its observations, and writes artifacts, claims, feedback, and provenance back to persistent memory.

An A-TXN is a task-scoped state machine:

$$A = (u, q, P^0, V^0, W, K, E, C, \mathcal{G}, o), \quad (4)$$

where u is identity, q the task, P^0 the policy epoch, V^0 a vector of source-version observations, W the bounded working set, K scoped capabilities, E execution records, C extracted or declared claims, G typed dependencies, and o the final outcome.

6.1 Version Vector and Attainable Consistency

For source systems $s_1, \dots, s_n$, AMOS records:

$$V^0 = \langle (s_j, v_j, \tau_j, \ell_j) \rangle_{j=1}^n, \quad (5)$$

where v_j is a snapshot, schema, metric, document, or policy version; τ_j is observation time; and ℓ_j is an optional lease or retention deadline. A version is not assumed to be globally simultaneous with other versions.

Each source is assigned one of three declared consistency classes:

- **C0 - observed:** a version and observation time are recorded, but the source cannot guarantee retention or monotonic reads.
- **C1 - validated:** the version can be re-read at commit and changes are detectable.
- **C2 - leased or immutable:** the version is retained or leased for the task and can be replayed during the declared interval.

Task policy specifies the minimum class for each required role. A high-risk report may require C1 policy, schema, and metric state plus C2 data; a low-risk exploration may tolerate C0 documents with a warning. If a source cannot satisfy the requirement, the runtime does not pretend to have a distributed snapshot: it returns `warning`, `needs_review`, or `reject`.

6.2 Transaction Protocol

```
A-TXN(request, identity, policy, budget):
  BEGIN:
    bind identity; read policy epoch P0; collect source vector V0
  ADMIT:
    derive required roles and consistency classes
    reject or downgrade if capabilities, budget, or versions are unavailable
  SELECT:
    filter by policy before candidate ranking
    reconcile logical keys, authority, effective time, and supersession
    choose role-complete working set W within context budget
  EXECUTE:
    issue task-scoped capabilities K
    verify each operation against W and policy; record results E
  VALIDATE:
    re-read policy epoch and all C1 versions; confirm C2 leases
    if a governing version changed: abort, repair, or require review
  COMMIT:
    extract or accept declared claims C; build dependency graph G
    atomically commit artifact, C, G, replay descriptors, and audit events
  INVALIDATE:
    on later source changes, traverse reverse dependencies and revalidate
```

The linearization point is the successful `COMMIT` in the AMOS store after external-version validation. This does not make external systems atomic with AMOS. It guarantees only that the committed record truthfully names the versions validated at publication and that any detected race prevents a final artifact. Changes after commit are handled by invalidation.

6.3 Role-complete Bounded Context

The proposal replaces the undefined goal of a “sufficient” context with an operational goal: *role completeness under a declared task schema*. A task parser produces mandatory roles $R(q)$ such as metric, schema, data state, policy, and supporting evidence.

Candidate selection solves:

$$\max_{W \subseteq \mathcal{M}} \sum_{i \in W} (rel_i + auth_i + time_i + active_i) - \lambda cost_i \quad (6)$$

$$\text{s.t. } permitted(i, P^0) \forall i \in W, \quad (7)$$

$$\sum_{i \in W} tokens(i) \leq B, \quad (8)$$

$$\forall r \in R(q), \exists i \in W : covers(i, r). \quad (9)$$

Mandatory-role failure is observable and produces a non-final outcome; it is not hidden by adding more loosely relevant text. Optional evidence fills remaining budget. Retrieval recall is evaluated against independent labels, and omitted candidate counts and uncovered roles are recorded.

Logical identity. Objects are grouped using source-qualified logical keys issued by an owning connector. Cross-source aliases, renames, merges, and splits require explicit alias or supersession edges with authority and effective intervals. Heuristic matches may propose edges but cannot silently establish governing identity. Multiple approved mappings for the same interval produce a conflict.

6.4 Bounded Verification Semantics

The SQL verifier does not claim general query equivalence. It supports a declared fragment: read-only relational SQL with approved aggregates, filters, joins, CTEs, grouping, and a bounded set of window constructs. Queries are parsed into relational expressions and checked against schema and metric constraints. Unsupported dialect features, UDFs, approximate functions, ambiguous NULL semantics, or plans beyond the fragment return `needs_review` or `reject` rather than an unsound equivalence claim.

Verifier evaluation measures valid acceptance, invalid rejection, false positive and false negative rates, repair success, and repair-induced violations on an independently authored corpus. The research claim is about useful enforcement within the declared fragment.

6.5 Claims, Support, and Invalidation

The unit of publication is a *policy-relevant claim*: a numeric, comparison, causal, forecast, or operational statement whose risk policy requires support or review. Structured reports declare claim spans and types. Free-form artifacts use an extractor with conservative thresholds; high-risk publication requires structured claims or human confirmation. Missed-claim rate is therefore a measured failure mode.

For claim c , AMOS stores typed edges to computations, data versions, semantic definitions, documents, execution hashes, and review decisions. Three properties remain distinct:

$$traceable(c) \not\Rightarrow correct(c) \not\Rightarrow causally_justified(c). \quad (10)$$

Provenance establishes traceability. Mechanical verifiers can establish limited computation-to-result consistency. Causal and high-impact operational conclusions remain review-gated unless supported by an approved design.

When source version v is superseded or revoked, reverse traversal finds dependent claims. Each claim is rechecked against its policy: it may remain valid, become stale, be revoked, or require review. The research question is whether this finer invalidation unit reduces review work and missed defects relative to artifact- or job-level lineage.

6.6 Replay Semantics

Replay is decomposed into measurable levels:

- **Computational replay:** rerun deterministic computation against retained inputs; compare result hashes or declared numeric tolerances.
- **Evidence reconstruction:** recover the versions, queries, documents, and review records used at publication.

- **Artifact regeneration:** regenerate charts or structured reports and compare declared semantic fields, not necessarily pixels or prose.
- **Trajectory replay:** reproduce the entire model interaction. This is recorded when possible but is not a core guarantee because providers and sampling can be nondeterministic.

An artifact is labeled replayable only at a named level. C2 versions are required for computational replay; C0/C1 sources can still support historical evidence reconstruction.

6.7 Security Guarantee Boundary

Retrieved text is treated as untrusted data, but labeling alone is not a security guarantee. Guarantees come from pre-context authorization, tenant-scoped indexes and caches, external capability checks, structured tool APIs, output validation, and commit-time policy revalidation. The model may follow an injected instruction; it still cannot acquire a capability or publish restricted provenance unless the external control plane authorizes it.

The threat model includes malicious retrieved content, malicious authorized users, compromised documents, cross-tenant index leakage, permission-revocation races, provenance expansion, multi-turn poisoning, inference through filtered results, and denial of service. Source-system compromise, aggregate inference below source-enforced privacy thresholds, and side channels outside the measured interfaces are documented residual risks.

7 Why a Memory Operating Layer?

The term is used narrowly, not as a claim that AMOS is a general-purpose operating system. AMOS interposes on memory movement and the scarce or privileged analytical resources attached to it:

Operating concept	AMOS resource or mechanism	Enforced property
Isolation domain	Tenant, identity, task, and policy epoch	No cross-domain context or capability use
Admission control	Required roles, consistency class, token and execution budgets	Unsafe or incomplete tasks do not silently proceed
Memory management	Persistent typed state and bounded active context	Prompt cost remains bounded as history grows
Capabilities	Task-scoped tool and publication permissions	Privileged operations are external to model text
Resource accounting	Tokens, rows, runtime, cost, review, provenance level	Auditable per-task consumption
Fault containment	Abort, repair, warning, review, reject, invalidation	Races and stale state do not become silent final output
Commit and recovery	Version validation, atomic evidence commit, idempotent retries	Publication has a defined state and recovery point

Table 5: The memory operating-layer analogy is grounded in context management, interposition, and resource control. Scheduling models across independent agents is outside this proposal.

If evaluation shows that these mechanisms provide no benefit beyond ordinary composition, the operating-layer framing is rejected and the result should be reported as governed analytical middleware.

8 AMOS Memory-Layer Architecture and Prototype Scope

The platform is organized into six cooperating planes. The A-TXN mechanism below is the transaction boundary across the middle four planes, while applications consume the resulting runtime through stable interfaces.

The existing reference prototype supplies a feasibility base: Python 3.11, SQLite FTS5 for typed memory, DuckDB for analytical execution, SQLGlot for parsing, FastAPI endpoints, versioned provenance records, and replay packages. The research implementation will add the A-TXN state machine, source-version adapters, optimistic validation, task-scoped capability tokens, claim dependency traversal, and the composed baseline.

Plane	Responsibility	Representative components
Source	Connect to governed systems without needlessly copying raw data	Warehouses, lakehouses, streams, catalogs, semantic layers, BI, documents, code, identity, policy
Memory	Maintain the persistent company model	Typed objects, bitemporal versions, ownership, authority, aliases, feedback, artifacts, provenance graph
Context	Construct bounded task state	Permission-first filtering, retrieval, reconciliation, compaction, required-role and uncertainty reporting
Agent runtime	Perform the analytical workload	Planner, model router, tools, budgets, retries, checkpoints, scheduling, recovery
Verification	Decide what can be accepted or published	Schema, metric, freshness, statistical, claim-support, policy, reproducibility, and review gates
Artifact and interaction	Deliver results and host applications	APIs, web, chat, notebooks, dashboards, alerts, reports, decks, email, and collaboration tools

Table 6: AMOS platform planes separate durable company memory and runtime control from replaceable models, tools, data systems, and user-facing applications.

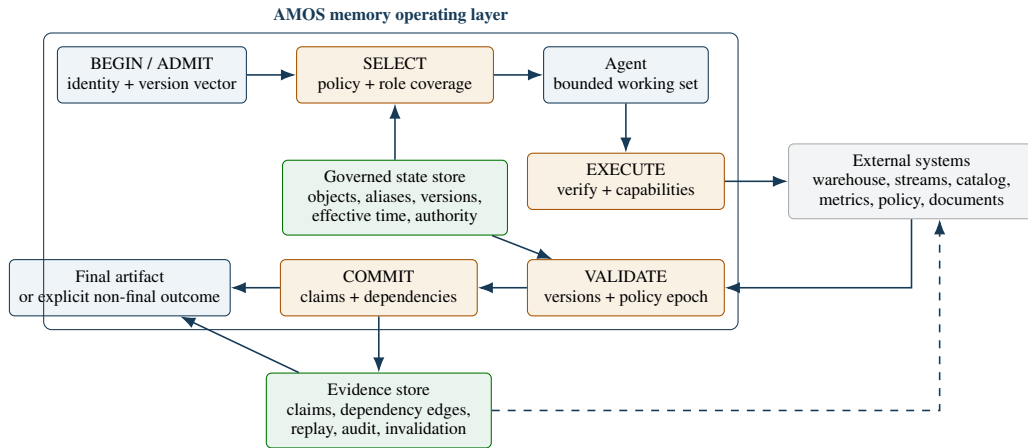


Figure 1: AMOS memory-layer architecture. A-TXN governs each memory projection and publication; dashed flow denotes later source-change events triggering claim-level invalidation.

The prototype will support three domains already represented in the artifact: payment failure, subscription churn, and warehouse quality. Each has independent schema, metric, temporal, permission, and review state. Storage and candidate-generation backends remain replaceable so the systems study can compare BM25, vector, and hybrid indexes under identical governance.

Preliminary engineering evidence. The current development artifact passes 85 local regression tests and seeded probes, and it has exercised indexed storage at 1,000,001 memory objects and 1,000,000 provenance edges. These results establish implementation feasibility and reveal a tail-latency bottleneck; they are not independent evidence for H1-H5 and will not be reused as confirmatory holdout data.

9 Experimental Methodology

The evaluation is preregistered before external labels or outputs are opened. Independent tasks, annotations, attacks, and human-study records are the statistical units; repeated model samples of the same task are not treated as independent evidence.

9.1 Systems and Fairness Controls

Arm	Purpose	Fairness constraint
Tool agent without persistent state	Lower bound for ordinary agent execution	Same model, tools, credentials, and task prompt
Long-context agent	Tests whether context size substitutes for state management	Same accessible state and maximum token budget
Permission-filtered RAG	Tests retrieval plus pre-context access control	Same candidate index and permission labels
Semantic/catalog agent	Tests current schema and metric grounding	Same definitions, tools, and version events
Catalog + lineage agent	Tests conventional metadata and job-level provenance	Native artifact/job lineage enabled
Production-faithful composed stack	Decisive novelty baseline	Agent orchestrator + filtered retrieval + semantic layer + catalog + policy engine + OpenLineage-shaped events + artifact store; no AMOS verifier or claim graph
AMOS	Full memory layer with A-TXN protocol	Same model, data, tools, context budget, retries, timeouts, and churn schedule
AMOS ablations	Isolate mechanisms	Remove one mechanism while preserving all others

Table 7: Comparison arms. Prompts and adapters will be published; systems receive matched information rather than intentionally missing inputs.

The composed stack is implemented before holdout release and reviewed by an engineer not responsible for AMOS scoring. It receives explicit orchestration for version reads and artifact storage, so the experiment tests whether the AMOS memory layer improves correctness or complexity beyond a credible integration, not whether other tools happen to omit a feature.

9.2 Independent Task Study

At least three analysts who did not implement AMOS will author 50 tasks in each of three domains, for 150 independent tasks. A different analyst reviews each task for realism and ambiguity; two annotators label required evidence, forbidden evidence, expected SQL or method, state-valid outcome, and review obligations before separate adjudication. Tasks are frozen and hashed before system tuning.

Each task is expressed with three neutral prompt frames and run on three independently frozen model families. The primary paired contrast, AMOS versus the matched agent without AMOS, requires 2,700 intended executions. Secondary arms run on the same task set under a declared sampling plan. Every intended execution remains in the denominator; provider, timeout, verifier, and malformed-output failures are preserved.

Workloads mix normal analysis with controlled evolution:

- schema additions, renames, removals, aliases, joins, and type changes;
- metric numerator, denominator, filter, unit, and effective-time changes;
- snapshot advancement, watermark lag, late data, retention, and missing partitions;
- document supersession, conflicting approvals, and malicious evidence;
- permission revocation, role change, policy-cache churn, and provenance expansion;
- unchanged benign tasks to measure false warnings, unnecessary review, and utility loss.

Churn events occur before selection, during execution, and immediately before commit to exercise the consistency protocol. A task is not designed around AMOS terminology and may require completion, clarification, warning, repair, rejection, or human review.

9.3 Outcomes

The preregistered primary endpoint is safe-and-correct completion, but it will never be reported alone. The paper will independently report:

- analytical answer and SQL correctness;
- schema, metric, temporal, and document-state correctness;
- permission violations and restricted-evidence exposure;
- retrieval recall, role coverage, stale selection, and omitted-evidence errors;
- unsupported-claim rate and review-obligation precision/recall;
- false rejection, false warning, repair success, and unnecessary-review rates;
- computational replay and evidence-reconstruction success;
- end-to-end and control-path latency, tokens, execution cost, and storage.

The independent task is the primary statistical unit. Paired differences use 10,000-replicate task-cluster bootstrap intervals. A logistic mixed-effects model includes task and author random intercepts and system, model, and prompt fixed effects. Holm correction applies to confirmatory secondary contrasts. Effect sizes, uncertainty intervals, per-task outcomes, and all failures will be released.

9.4 Verifier Study

A separately authored 400-query corpus contains 200 valid and 200 invalid cases across aliases, derived columns, CTEs, nested and correlated subqueries, joins, windows, grouping sets, conditional aggregates, NULL behavior, dialects, stale fields, missing metric predicates, comments, obfuscation, writes, DDL, exports, and unsafe functions. Two annotators label each case and adjudicate disagreements.

Primary metrics are valid acceptance, invalid rejection, false positive and false negative rates, repair success, and repair-induced violation rate. Headline use of the verifier requires at least 0.95 valid acceptance, 0.95 invalid rejection, and at most 0.02 repair-induced violations. Failing a construct family narrows the supported SQL fragment; it does not permit task-specific patching after unblinding.

9.5 Claim and Human Audit Studies

The claim corpus contains at least 120 raw reports, notebooks, slides, chart annotations, dashboard texts, table cells, and fragments with at least 600 adjudicated claims. Splits are by source artifact and author. Regex, prompted LLM, structured-output LLM, and a competitive supervised model are compared on exact-span F1, claim-type macro-F1, review-obligation F1, unsupported-claim F1, and evidence-link F1.

A within-participant, randomized, counterbalanced audit study recruits at least 36 technically qualified participants. Participants inspect seeded reports under either claim-level dependencies or ordinary citations plus job logs. Primary outcomes are defect-detection rate and audit time; secondary outcomes are false alarms, confidence, replay success, and usability. H3 requires improvement in at least one primary outcome without credible degradation in the other.

9.6 Security Study

At least three independent red-teamers create 120 attacks across direct and indirect prompt injection, cross-user leakage, malicious authoritative sources, permission revocation, provenance expansion, timing or filtered-result inference, and multi-turn memory poisoning. Protected assets, attacker knowledge, allowed credentials, and success criteria are frozen in advance.

The primary security outcome is attack success rate, paired with safe-and-correct completion on matched benign tasks. A confirmed permission leak fails the headline security gate. The study distinguishes model compromise from system compromise: an injected model response counts as contained if external capabilities and publication gates prevent unauthorized effects.

9.7 Systems, Scale, Churn, and Recovery

Systems experiments use 10,000, 100,000, and 1,000,000 memory objects with matched provenance graphs, plus sensitivity runs at higher fan-out where hardware permits. Workloads vary context budget, candidate engine, 1-64 concurrent readers, mixed reads/writes, policy and schema churn, invalidation fan-out, cache state, source delay, connector failure, and store restart.

Measurements include p50/p95/p99 retrieval and transaction latency, throughput, CPU, memory, storage amplification, token use, cost, stale-commit rate, version-validation aborts, retry amplification, invalidation completion, recovery time, and correctness after recovery. H4 fails if AMOS exceeds 25% matched control-path overhead without a significant state-validity or audit benefit, or if correctness drops by more than five points as persistent state scales while context budget remains fixed.

9.8 Ablations

The ablation matrix removes one mechanism at a time:

- no publish-time version or policy revalidation;
- no effective-time or authority reconciliation;
- post-retrieval permission filtering instead of index pushdown;
- unconstrained top- k retrieval instead of mandatory role coverage;
- artifact-level lineage instead of claim dependencies;
- no reverse invalidation traversal;
- C0 observation only instead of C1/C2 consistency classes.

The purpose is to identify which part of A-TXN causes any observed benefit. If the full system improves but no mechanism explains the effect, the proposal will report the result as unresolved rather than attributing it to the abstraction.

10 Success, Failure, and Interpretation

Result pattern	Interpretation	Required action
AMOS beats weak agents but not composed stack	Integration, not a new abstraction, explains the gain	Reject H2; position artifact as reference integration
Correctness improves but false rejects or review burden exceed gates	Safety-utility tradeoff is unacceptable	Narrow policies or reject H1 for general use
Claim views improve detection but not time	Auditability benefit with operator cost	Report both; do not claim reduced audit cost
Overhead exceeds 25% with meaningful safety gain	Mechanism works but is not yet practical	Publish cost boundary; narrow deployment scope
No effect on independent tasks	Seeded development tasks overfit the design	Report null result; release benchmark and failure analysis
Any permission leak	Security boundary failed	Remove security headline; repair only on a fresh holdout

Table 8: Interpretation rules prevent redefining success after results are known.

No task-specific prompt, verifier, retrieval, or scoring changes are permitted after holdout labels are opened. Bugs discovered after unblinding are fixed only for a new independently authored confirmatory set. Local seeded results remain development evidence.

11 Research Validation Plan and Resources

The product roadmap in Table 2 is gated by customer and operational evidence. In parallel, the following nine-month research program tests whether the underlying systems thesis is defensible; it does not substitute for production discovery.

Month	Deliverable	Go / no-go criterion
0-1	Final A-TXN specification, consistency classes, threat model, and measurement harness	Every core term has an executable test oracle
2-3	Version adapters, role-complete selection, validation/commit, claim dependencies	Development races are detected; atomic evidence commit survives fault injection
3-4	Production-faithful composed baseline and matched adapters	Independent review confirms equivalent sources, models, budgets, and tools
4-5	Verifier and claim pipelines; pilot only on development data	At least 0.90 development acceptance/rejection or scope narrows before freeze
5-6	Ethics and privacy review, participant plan, frozen configuration and preregistration	No holdout labels or outputs opened before all gates pass
6-8	Independent tasks, claim corpus, human audit, red team, and systems runs	Missing executions and deviations preserved; no post-hoc tuning
8-9	Statistical analysis, artifact audit, paper, and public release	Every claim maps to immutable raw evidence and regeneration scripts

Table 9: Implementation and research schedule.

People. The minimum plan requires one primary systems implementer; at least three independent task authors/reviewers; two annotators per task or artifact plus separate adjudication; three red-teamers; one study custodian; and 36 audit-study participants. Roles may overlap only across different items without violating the independence boundary.

Compute and budget. The systems study assumes one reproducible 32-core server with at least 128 GB RAM and 2 TB SSD, plus object storage for raw traces. The primary model study contains 2,700 intended executions before secondary arms; a provider and participant budget of approximately USD 10,000-20,000 is reserved and frozen before collection. Exact prices, models, retries, timeouts, and hardware are archived with results.

Fallbacks. If a live external product cannot be evaluated with exportable evidence, the secondary comparison uses a production-faithful self-hosted composition and is labeled accordingly. If claim extraction cannot meet recall gates, high-risk publication requires structured claim declarations. If general SQL verification fails, the supported fragment is narrowed. If C2 retention is unavailable, claims are limited to evidence reconstruction rather than computational replay. None of these fallbacks changes the H1/H2 evaluation after freeze.

12 Reproducibility, Ethics, and Artifact Plan

The repository already contains a frozen external-evaluation schema, evidence validators, scenario contracts, and machine-readable development results. The confirmatory release will add:

- immutable source revision, dependency manifests, container or environment records, and hardware description;
- hashed task, annotation, model, baseline, prompt, timeout, retry, and price configurations;
- anonymized holdout tasks and raw outputs where data-use agreements permit;
- all failures, exclusions, protocol deviations, and missing executions;
- scripts that regenerate every figure and table from raw records;
- a claim-to-evidence manifest for the final paper.

Human studies begin only after ethics, consent, privacy, compensation, exclusion, and data-retention rules are approved. Participant identities remain private and separate from public pseudonymous records. Proprietary product exports and sensitive data are retained only in approved access-controlled archives; public releases contain hashed manifests and permitted redactions.

13 Expected Contributions

The project is expected to contribute:

1. **A memory-layer abstraction:** persistent typed analytical memory, bounded active-context projections, and explicit read/write authority for continuous data analytics agents.
2. **A transaction protocol:** A-TXN, with consistency classes, optimistic version validation, capability-scoped execution, and a publication linearization point.
3. **Two coupled memory mechanisms:** role-complete bounded context selection and claim-level dependency/invalidation semantics, both evaluated independently through ablations.
4. **A reference implementation and composed baseline:** matched enough to test whether unification adds value beyond ordinary integration.
5. **Independent evidence:** task, verifier, claim, human-audit, red-team, and scaling studies with preregistered failure criteria.
6. **A reusable artifact:** schemas and tools for measuring state-related failures in long-running analytical agents, even if the central hypothesis is rejected.

The strongest possible result is not that AMOS passes its own invariants. It is that the memory layer produces a reproducible improvement over a credible composed alternative and that the improvement can be attributed to its context, transaction, and dependency mechanisms at measured cost. A null result is also informative: it would show that conventional composition is sufficient or that the proposed safety-utility tradeoff is not worthwhile.

14 Conclusion

AMOS is designed to become the persistent analytical intelligence of a company: an operating layer in company memory that understands its data, metrics, ontology, policies, history, authority, decisions, and preferred ways of communicating, then continuously performs the complete work of data analysis. Teams and external developers build analytical applications on that shared substrate. Conversational querying is one such application, and a model such as Google Gemma 4 is one replaceable processor; neither defines the product boundary.

The research program deliberately narrows this broad product vision to a falsifiable systems question. AMOS keeps persistent typed analytical state outside the model, constructs bounded governed context for each task, and preserves corrections and evidence across tasks. A-TXN makes the layer enforceable through version vectors, role-complete context selection, scoped capabilities, optimistic commit validation, and claim dependencies.

The research program is designed to reject the idea if it does not outperform a production-faithful composed stack, if its verifier rejects valid work, if its audit interface does not help reviewers, if its security boundary leaks, or if its overhead overwhelms the benefit. That combination of mechanism, comparison, and failure criteria is the basis on which AMOS should be judged as systems research.

References

- [1] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [2] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 2020.

- [3] Nelson F. Liu et al. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024.
- [4] Shunyu Yao et al. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.
- [5] Timo Schick et al. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- [6] Tao Yu et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL. *Proceedings of EMNLP*, 2018.
- [7] Jinyang Li et al. Can LLM already serve as a database interface? A BIG Bench for large-scale database grounded text-to-SQLs. *arXiv preprint arXiv:2305.03111*, 2023.
- [8] Fangyu Lei et al. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. *International Conference on Learning Representations*, 2025.
- [9] Tyler Akidau et al. MillWheel: Fault-tolerant stream processing at internet scale. *Proceedings of the VLDB Endowment*, 2013.
- [10] Tyler Akidau et al. The Dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 2015.
- [11] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache Flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, 2015.
- [12] Michael Armbrust et al. Delta Lake: High-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment*, 2020.
- [13] Luc Moreau and Paolo Missier, editors. PROV-DM: The PROV data model. W3C Recommendation, 2013.
- [14] OpenLineage Project. OpenLineage: An open standard for lineage metadata collection. <https://openlineage.io/>, accessed 2026.
- [15] Sebastian Schelter et al. Unit testing data with Deequ. *Proceedings of SIGMOD*, 2019.
- [16] Eric Caveness et al. TensorFlow Data Validation: Data analysis and validation in continuous ML pipelines. *Proceedings of SIGMOD*, 2020.
- [17] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.
- [18] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. *arXiv preprint arXiv:2312.14197*, 2023.
- [19] Joon Sung Park et al. Generative agents: Interactive simulacra of human behavior. *Proceedings of the ACM Symposium on User Interface Software and Technology*, 2023.
- [20] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 2023.
- [21] Leonardo Murta, Vanessa Braganholo, Fernando Chirigati, David Koop, and Juliana Freire. noWorkflow: Capturing and analyzing provenance of scripts. *Provenance and Annotation of Data and Processes*, 2014.
- [22] Timothy McPhillips et al. YesWorkflow: A user-oriented, language-independent tool for recovering workflow information from scripts. *International Journal of Digital Curation*, 10(1):298–313, 2015.